Technical Design Document: InfoBridge PoC v0.1

DSI 2026, Project 9A: Bringing Fragmented Information Together
Team: The North Remembers · **Anchor NGO:** Hyderabad Runners
Sprint 2 deliverable (27 September – 10 October 2026) · Version 0.1

> Sprint 2 checkpoint: "*A first working technical PoC exists, even if crude or incomplete.*" InfoBridge v0.1 ingests five source formats, retrieves evidence, produces answers with validated, fact-level citations, and is scored automatically against a synthetic gold standard. The baseline numbers are in §6, and §7 lists what they show is still missing.

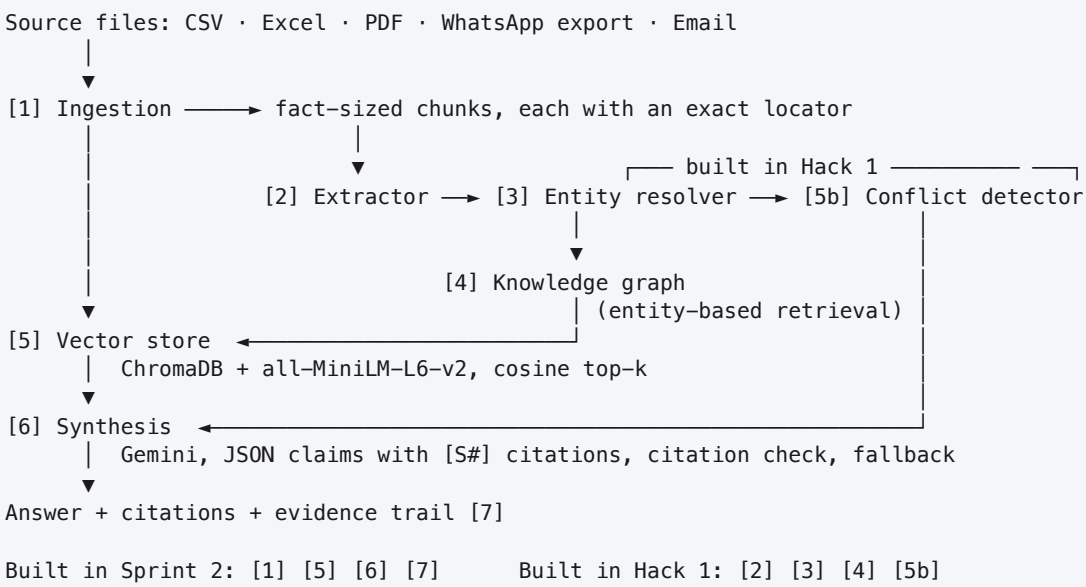
---

1. Scope of this version

	Sprint 2 (v0.1 foundations)	Hack 1 (core build, ~60% of project)	Sprint 3+
Pull together	Multi-format ingestion, vector retrieval, LLM synthesis	Fact extraction (tables + LLM for text); knowledge graph; entity-based retrieval	More question types; one open-data scenario
Show provenance	Fact-sized chunks with exact locators, citation validation, source → claim trail	Full trail: source → fact → resolved entity → claim; demo app	–
Flag conflicts	Measured only: does a plain LLM notice conflicts unaided?	Rule-based conflict detector; conflict-aware answers	Harder conflict cases; resolution suggestions evaluated
Entity resolution	None (that is the point of the baseline)	Phone/email keys, name rules, fuzzy matching	Phonetic matching, ambiguous names, confidence

Sprint 2 lays the foundations: the project skeleton with fixed component interfaces, the synthetic dataset with answer keys, and the simplest pipeline that answers with citations. Hack 1 then builds the core components on top of it. That way each later component has a baseline number it has to improve. The full sprint-by-sprint plan is in `docs/roadmap.md`.

2. Architecture



Component status

#	Component	Module	Input → Output	Status
1	Document Ingestion	`ingestion.py`	files → `Document`, `Chunk[]`	**Done**
2	Information Extractor	`extraction.py`	`Chunk[]` → `Fact[]`	Interface + plan
3	Entity Resolver	`resolution.py`	`Fact[]` → `Entity[]`	Interface + plan; scoring metric done
4	Knowledge Graph Builder	`graph.py`	entities, facts, conflicts → graph	Interface + plan
5	Vector store / retrieval	`retrieval.py`	question → `RetrievedChunk[]`	**Done**
5b	Conflict Detector	`conflicts.py`	entities, facts → `Conflict[]`	Interface + plan; scoring metric done
6	Synthesis Engine	`synthesis.py`	question + chunks → `Answer`	**Baseline done**
7	Provenance Tracker	`provenance.py`	`Answer` → evidence trail	**Partial** (source → chunk → retrieval → claim)
–	Evaluation	`evaluation.py`	answers + `gold.json` → metrics	**Done**
–	Fact vocabulary	`schema.py`	attribute names + spreadsheet column mappings	**Done**
–	Demo app	`app.py`	question → answer, citations, conflicts, trail	**Skeleton** (on the v0.1 pipeline)

Data contracts (`models.py`)

Every component consumes and produces these typed objects (Pydantic). They are the component APIs from the Design Doc, written as code.

- **`Chunk`**: `chunk_id = "{doc_id}#{locator}"`, text, metadata. This is the smallest unit that can be cited.
- **`Fact`**: (entity_mention, attribute, normalised value, raw value, `chunk_id`, extractor, confidence).

- **`Entity`**: canonical name, type, `Mention[]` (text + `chunk_id`), `match_evidence` (why mentions were merged).
- **`Conflict`**: entity, attribute, `ConflictValue[]` (each with its `chunk_id`), severity, suggestion.
- **`Answer`**: verdict, summary, `Claim[]` (text + citation labels + `unsupported` flag), `Citation[]` (label → chunk, locator, verbatim quote), conflicts, retrieved chunks, `fallback` flag, latency.

Provenance rule: anything derived from a source carries a `chunk_id`, and a chunk carries a `doc_id` and a locator. Any output therefore resolves to an exact row, message or paragraph.

3. Implementation

3.1 Ingestion: fact-sized, locatable chunks

Most RAG systems chunk by token windows. We chunk by the source's natural unit instead, so that a citation points to one fact rather than to a whole document:

Format	Chunk unit	Locator example	Notes
CSV / Excel	one row	`members.csv#row=5`	Spreadsheet numbering (header = row 1), so it matches what the coordinator sees in Google Sheets. Row fields kept in metadata.
WhatsApp export	one message	`whatsapp_coach_group.txt#msg=3`	Parses `dd/mm/yyyy, hh:mm – Sender: text`; joins multi-line messages; keeps sender and timestamp.
PDF	one text block / paragraph	`programme_rules.pdf#page=1&para=2`	PyMuPDF `get_text("blocks")`, sorted into reading order.
Email / text	one paragraph	`email_long_run_route.txt#para=3`	Headers (From/Date/Subject) go into metadata and are prefixed to each chunk for retrieval context.

The format is detected automatically, and a WhatsApp export is recognised from its first line.

3.2 Retrieval

ChromaDB (in-memory) with **all-MiniLM-L6-v2** embeddings, cosine similarity, top-k = 8. Chroma runs this model through ONNX, so we use the Design Doc's model without a PyTorch dependency.

3.3 Synthesis (naive-RAG baseline)

1. The retrieved chunks are shown to the LLM as numbered sources: `[S3] (members.csv, row=5) …`.
2. The system prompt requires every claim to cite at least one label and forbids outside knowledge. It also tells the model to state disagreements between sources, so the baseline is a reasonable prompt and not deliberately weakened.
3. The model returns JSON: `verdict` (YES / NO / UNCERTAIN / null), a cited `summary`, and `claims[]` with citations.
4. **Citation validation:** every label is checked against what was actually retrieved. Claims citing nothing, or a label that does not exist, are flagged `unsupported`, which catches hallucinated citations.
5. **Fallback (Design Doc §3.5):** if the best retrieval score is below 0.25, the system does not call the LLM. It returns "not enough reliable evidence" and lists the closest sources to check.

LLM: Gemini (`gemini-2.5-flash` by default, configurable), temperature 0, JSON response mode. An **extractive backend** returns the top-5 chunks verbatim as claims. It needs no API key and serves as a retrieval-only lower bound.

3.4 Provenance trail

For each claim, `python -m infobridge trail …` outputs: source document (path, format) → chunk (locator, exact text) → retrieval (rank, similarity) → claim. Hack 1 adds the extraction and entity-resolution steps between chunk and claim.

3.5 Hack 1 readiness

The last Sprint 2 activity is preparing the component for a focused Hack 1 build. Since Hack 1 has to deliver about 60% of the project in half a day, everything that isn't the core logic itself is in place beforehand:

Prepared	What it gives the Hack 1 team
Fact vocabulary (`schema.py`)	One list of 31 attribute names, plus column → attribute mappings for every spreadsheet, so the extractor, resolver and conflict detector agree on names.
Fixtures (`tests/fixtures/case_03_hack1.json`)	22 hand-made facts for case 3, the expected entity groupings and the 2 expected conflicts. The resolver and conflict detector can be built and tested without waiting for the extractor.
Acceptance tests (`tests/test_hack1_contracts.py`)	5 tests that currently fail by design (`xfail`), one set per Hack 1 task. A task is done when its tests pass.
LLM response cache (`results/cache/`)	Identical Gemini calls are answered from disk, which protects the free-tier quota during repeated runs. Turned off when measuring latency.
Demo app skeleton (`app.py`)	A one-page Streamlit app showing the verdict, claims with citations, sources, conflict cards and the evidence trail. It already runs on the v0.1 pipeline and picks up conflicts automatically once Hack 1 fills `Answer.conflicts`. It is a demo harness, not a product frontend.
Feedback log (`docs/hack1_feedback_log.md`)	Template for recording failures and questions during the hack.

4. Synthetic evaluation dataset

The project brief asks for "synthetic multi-document cases with facts distributed across sources." For each case we wrote the gold standard first, deciding the facts, name variants, decoys and conflicts, and then wrote the source documents to contain them.

Case	Sources (formats)	Question	What it tests
1 – Simple	events sheet (CSV), organiser email (text)	Details of the 25 Oct Monthly Long Run	Basic two-source synthesis, no conflicts
2 – Medium	volunteer roster (Excel), AS3 WhatsApp group, first-aid attendance (PDF)	Who is at Aid Station 3 and are they first-aid trained?	Name variants across formats (*Srinivas Rao Kumar / Srini / S. R. Kumar*; *Mohammed Farhan / Farhan bhai*); a decoy (*Srinivas Reddy*, a different person); a gap (Farhan is untrained)
3 – Conflict	members (CSV), registrations (CSV), coach WhatsApp group, programme rules (PDF)	Is Ravi Sharma eligible for HMTP Batch 2026B?	Three name forms (*Ravi Sharma / R. Sharma / Ravi S.*); a decoy (*Ravi Shankar*: lapsed membership, late registration); **two planted conflicts**: medical clearance date (sheet 15 Mar vs doctor's message 22 Jun) and batch start date (registration sheet 1 Nov vs rules PDF 8 Nov); expected verdict **YES**

Each `gold.json` contains:

- **facts**: accepted surface forms plus the chunk IDs that support each fact
- **entities**: every mention and its chunk ID, for entity-resolution F1
- **conflicts**: both values, their surface forms and sources

- **queries**: question, expected verdict, relevant fact and conflict IDs

A test (`tests/test_gold.py`) checks that every chunk ID in the gold files exists and that every entity mention appears in its chunk.

5. Evaluation criteria

Metric	Definition	Design Doc metric
Retrieval recall@k	Share of gold facts with ≥1 supporting chunk in top-k	(diagnostic)
Fact recall	Share of gold facts stated in the answer	Synthesis completeness
Citation validity	Share of cited labels that resolve to a retrieved chunk	Citation accuracy (hallucination check)
Citation accuracy	Of stated facts, share whose claim cites a gold supporting chunk	Citation accuracy
Unsupported claims	Share of claims without a valid citation	Citation accuracy
Conflict recall (text)	Share of planted conflicts where the answer states both values	Conflict detection recall (unaided LLM)
Conflict recall (structured)	Share of planted conflicts in `Answer.conflicts`	Conflict detection recall
Verdict accuracy	Exact match on YES/NO/UNCERTAIN	Decision correctness
Entity resolution F1	Pairwise F1 over (mention, chunk) pairs	Entity resolution F1 (Hack 1)
Latency	Seconds from question to answer	Latency

Fact and conflict matching use keywords: the gold file lists accepted surface forms. This is reproducible and free, but it can miss paraphrases, so it understates fact recall for LLM answers. We plan to add a manual or LLM-judge check for the final evaluation. Response coherence (a human rubric) is not measured yet.

Targets (see `docs/roadmap.md`):

Metric	v0.1 baseline	Hack 1	v0.2 (Sprint 3)	v1.0
Retrieval recall (case 3)	0.57	≥ 0.90	≥ 0.90	≥ 0.95
Structured conflict recall	0.00	1.00 (C1 + C2)	≥ 0.90 on all cases	≥ 0.90
False conflicts (cases 1–2)	–	0	–	–
Entity resolution F1	–	≥ 0.85	≥ 0.90	≥ 0.90
Citation validity	1.00	1.00	1.00	1.00
Verdict accuracy (case 3)	TBD	1.00	≥ 0.90 on all cases	≥ 0.90

6. Baseline results

Command: `python -m infobridge eval --backend <backend> --tag <tag>`. Full outputs, including every answer, are in `results/` .

6.1 Retrieval-only baseline (extractive, top-5 chunks as the answer)

Case	Retr. recall	Fact recall	Cit. validity	Cit. accuracy	Unsupported	Conflict (text)	Conflict (struct.)	Verdict	Latency
1 – Simple	1.00	1.00	1.00	1.00	0.00	–	–	–	0.12 s

Case	Retr. recall	Fact recall	Cit. validity	Cit. accuracy	Unsupported	Conflict (text)	Conflict (struct.)	Verdict	Latency
2 – Medium	0.67	0.50	1.00	0.67	0.00	–	–	–	0.18 s
3 – Conflict	0.57	0.14	1.00	1.00	0.00	0.00	0.00	0.00	0.10 s
Mean	**0.75**	**0.55**	**1.00**	**0.89**	**0.00**	**0.00**	**0.00**	**0.00**	**0.14 s**

6.2 Naive-RAG baseline (Gemini)

> **Pending:** needs a Gemini API key. Run `python scripts/finalize_baseline.py`, which runs the evaluation, inserts the table here and rebuilds the PDF.

7. What worked and what didn't

Worked

- Ingesting five formats into one chunk model with exact locators. Every citation resolves to a row, message or paragraph (citation validity 1.00).
- The simple, consistent two-source case is answered completely.
- Fallback mode: an off-topic question ("What is the capital of Peru?") is refused, with the closest sources suggested instead of a guess.
- The evaluation harness runs end to end against the gold standard in under a second, without an API key.

Didn't work (and why: these set the Hack 1 priorities)

1. **Spreadsheet rows are nearly invisible to dense retrieval.** On case 3, every one of the 11 CSV rows ranks 15th or lower out of 29 chunks. Ravi Sharma's own membership row ranks **20th**, and his registration row ("*R. Sharma*") ranks **24th**. `key=value` rows embed poorly next to natural-language messages, so retrieval recall is 0.57 and the membership and registration facts never reach the LLM. Any LLM answer is capped by this. **Fix (Hack 1):** entity-based lookup through the knowledge graph (fetch every chunk that mentions the resolved entity), plus a keyword (BM25) channel for names and IDs.
2. **The decoy outranks the target.** Within the top-8, the message about **Ravi Shankar's** lapsed membership ranks #4. In the members sheet, Ravi Shankar's row (#15) also outranks Ravi Sharma's (#20). A naive system could wrongly conclude Ravi Sharma is ineligible. **Fix (Hack 1):** entity resolution using strong keys (phone/email) to keep *Ravi Shankar* separate, and filtering retrieved evidence to the resolved entity.
3. **Conflicts are invisible when only one side is retrieved.** For the clearance-date conflict, Dr. Rao's message (22 June) is retrieved (#5), but the members-sheet value (15 March, #20) is not, so the LLM cannot notice the contradiction. Structured conflict recall is 0 by construction. **Fix (Hack 1):** a deterministic conflict detector over all extracted facts, which works on facts rather than on retrieved chunks, so retrieval misses don't hide conflicts.
4. **Keyword-based fact matching** under-counts paraphrased answers (see §5).

8. Changes from the Sprint 1 Design Doc

Design Doc	v0.1	Reason
Gemini 1.5 Pro	Gemini 2.5 Flash (configurable)	1.5 Pro has been retired; Flash is on the free tier

Design Doc	v0.1	Reason
LangChain orchestration	Thin wrapper (~30 lines)	Makes prompts and citation parsing easy to inspect and debug at PoC scale; can be revisited
spaCy NER as the extractor	Schema mapping for tables + LLM JSON extraction for text; spaCy as a mention cross-check	NER finds "Ravi" and "June 22" but not that one is the other's *clearance date*, and conflict detection needs attribute-level facts
sentence-transformers	Same model (all-MiniLM-L6-v2) via Chroma's ONNX runtime	No PyTorch install; identical embeddings

9. Next steps

Hack 1 (10 October): build the core, about 60% of the project. The goal is the full Design Doc pipeline running end to end on all 3 cases, shown in a working demo video. Detailed task specs are in `docs/sprint2_status_and_hack1_plan.md`.

Work	Done when
Fact extraction: spreadsheet columns → facts; Gemini extraction for chats/PDFs/emails; normalise dates and phone numbers	Facts for all 3 cases
Entity resolution: phone/email keys → name rules (initials, nicknames) → fuzzy matching; never merge different surnames	ER F1 ≥ 0.85; *Ravi Shankar* and *Srinivas Reddy* kept separate
Conflict detection: group facts by (entity, attribute), compare normalised values, add severity and suggestion	C1 and C2 detected; nothing flagged in cases 1–2
Knowledge graph + entity-based retrieval: NetworkX graph with provenance edges; fetch every chunk about the resolved entity	Case 3 retrieval recall ≥ 0.90
Conflict-aware answer + full evidence trail	Verdict YES with both conflicts and their sources
Demo app (one-page Streamlit, a demo harness rather than a product frontend) + **evaluation** + **demo video**	Before/after table; video recorded

Later sprints (details in `docs/roadmap.md`):

- **Sprint 3 (v0.2):** act on Hack 1 feedback; 6+ test cases with new question types; one open-data scenario; better entity matching; LLM-based answer scoring.
- **Sprint 4 (v0.9):** systematic and edge-case testing, robustness, latency, failure analysis.
- **Hack 2 / Sprint 5 (v1.0):** polish, final evaluation, slides, poster, video.

10. Questions for instructor feedback

1. Is keyword-based fact matching acceptable for intermediate evaluations, provided the final evaluation adds a human or LLM-judge check?
2. Should conflict *resolution* (suggesting which value is correct) be evaluated, or only conflict *detection*?
3. For the open-data sources (OGD, WHO, OpenAlex), is one real cross-source scenario enough to demonstrate generality, given that the controlled evaluation is synthetic?

Appendix: running the PoC

```
uv venv --python 3.10 .venv && uv pip install --python .venv/bin/python -e ".[dev]"
cp .env.example .env    # add GEMINI_API_KEY
python -m infobridge ask data/synthetic/case_03_conflict/sources "Is Ravi Sharma eligible for the
Half Marathon Training Programme Batch 2026B?"
python -m infobridge eval --backend gemini --tag baseline_v0.1
streamlit run app.py   # demo app (needs: uv pip install streamlit)
pytest                 # 19 pass; 5 Hack 1 acceptance tests are expected to fail until built
```